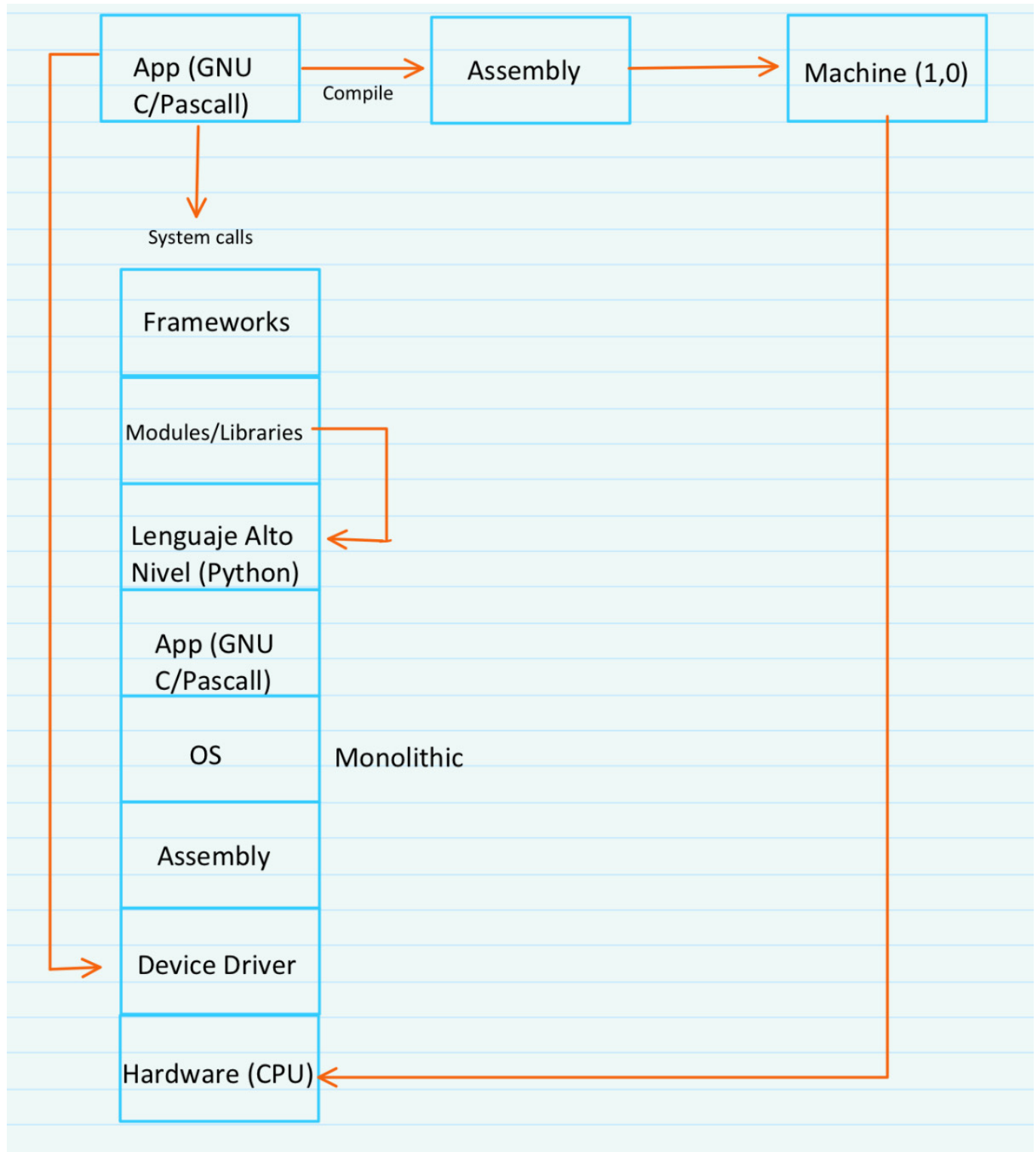


Apuntes de Clase

¿Qué es un computador?

- **Hardware:** son los transistores, tarjetas y movimiento de datos mediante pulsos
- **Assembler:** lenguaje que ejecuta instrucciones propias de la arquitectura
- **Device Drivers:** intermediario entre el hardware y el lenguaje ensamblador
- **OS (Sistema Operativo):** establece ciertas interfaces para que los programas interactúen con el hardware. Suele ser monolítico y pesado ya que viene con apps instaladas, además que siempre se le debe de pedir permiso para ejecutar
- **Apps:** estas compilan y generan un script ensamblador, que luego pasa a generarse como lenguaje máquina
- **System calls:** son operaciones de muy bajo nivel, que debo de pedirle al OS que las ejecute por mí
- **Lenguajes de Alto Nivel:** son lenguajes cuyas instrucciones y paradigmas de programación son mucho más simples. No obstante, se pierde el control que se tiene sobre el computador, generando que bajemos la eficiencia de la ejecución del programa
- **Modules/Libraries:** son bloques de código escritos en lenguajes de alto nivel, que se pueden reutilizar
- **Frameworks:** son estructuras ya definidas que te dan las reglas de cómo hacer una aplicación. Facilitando aún más a los programadores

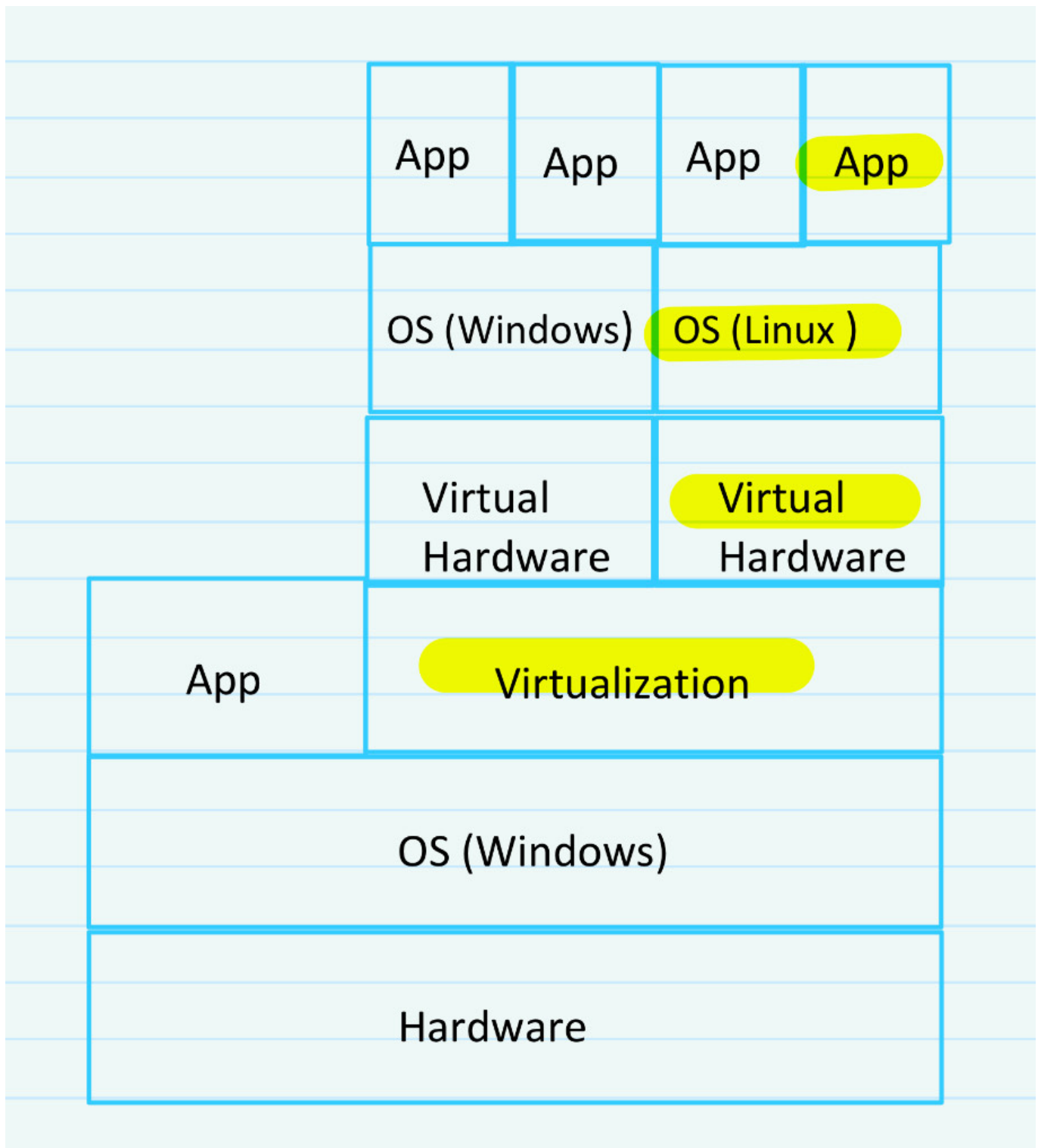


Evolución

Lo malo de esto, es que había un **problema** porque si la computadora utilizaba Windows y se le solicitaba usar Linux, entonces se debía desinstalar un OS para instalar el otro. Entonces apareció dual boot.

- **Dual boot:** este permitía tener más de un OS en la máquina, sin embargo tenía como problema que no se podían correr simultáneamente
- **Input/Output streams:** son los que mueven o jalan información entre diferentes dispositivos

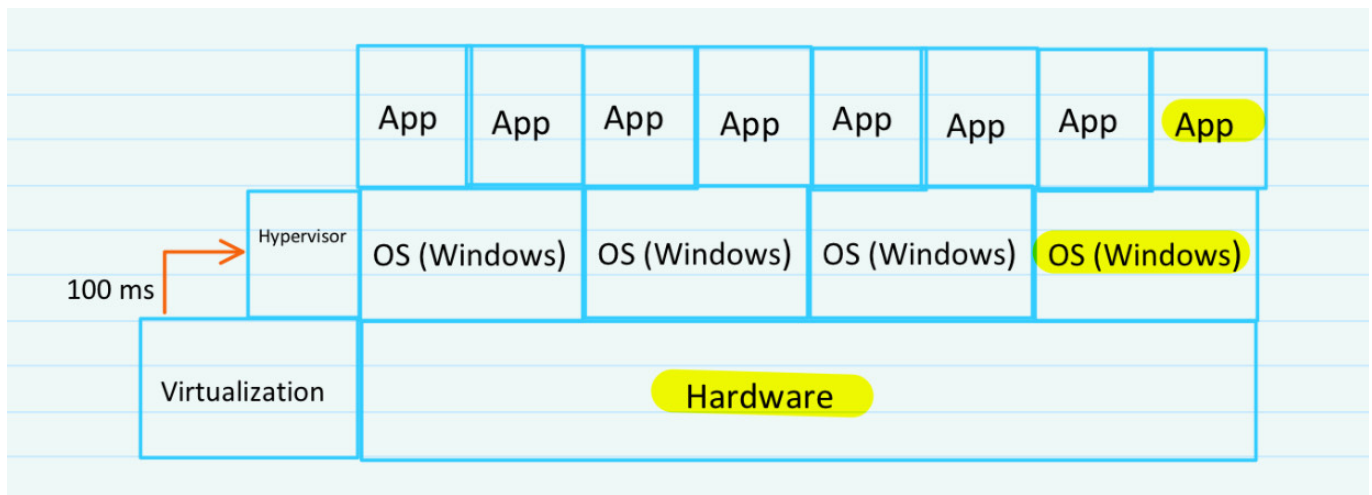
- **Virtualization:** permitía tener un hardware virtual para correr múltiples OS simultáneamente, la primera aplicación que hizo esto fue **VMWare**
- **Scheduling:** le da cierta cantidad de tiempo a una aplicación para ser la dueña del procesador y ejecutar instrucciones. Entonces el virtualizador va ejecutando sus distintos hardware virtuales



En esta imagen podemos apreciar cómo hay un scheduling, donde le toca al virtualizador, ahí se aplica otro scheduling, donde le toca al virtual hardware con OS Linux y se aplica otro scheduling, donde le toca a una app en específico

Luego, se implementó la virtualización desde el hardware propio de las computadoras, para ahorrarse varios hardware virtuales de distintos OS

- **Hypervisor:** OS sumamente pequeño, donde su función es realizar el scheduling de máquinas virtuales

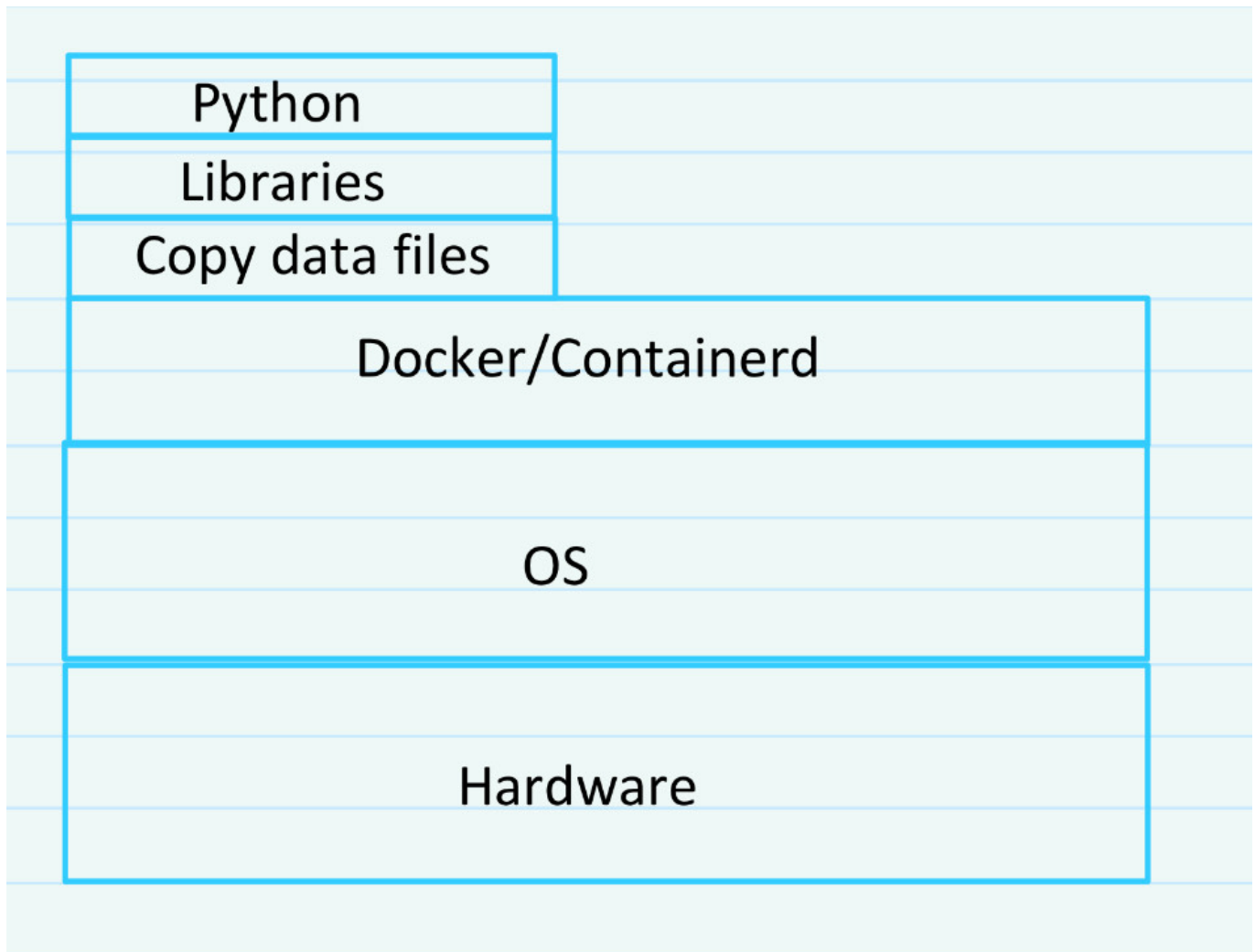


En esta imagen podemos apreciar cómo la virtualización activa al Hypervisor cada 100 ms para que realice el scheduling, de una manera más eficiente

- **Context Switch:** es el cambio de OS que se da en cada hypervisor

Debido a que en este modelo, se debe estar recurrentemente haciendo context switch en cada OS, se volvía muy ineficiente. Por lo que se creó una solución más sostenible, docker

- **Docker:** crea contenedores con lo único necesario, trabajando en capas y de esta manera ahorra espacio, permite que el OS sea monolítico y que el context switch se reduzca
- **Kubernetes:** es un orquestador de contenedores, el cual tiene múltiples servidores y siempre busca el mejor para el respectivo contenedor. Además, que si algo se cae, el mismo se encarga de que vuelva a correr



Proyecto

- El profe nos entrega un proyecto base para guiarnos y desarrollar el proyecto
- En el proyecto hay un README, donde se nos guía paso a paso cómo instalar los requerimientos
- **Docker hub**: es un repositorio de imágenes de la comunidad, ahí se puede obtener Elasticsearch, Python y más
- **Pods**: es un contenedor que puede tener más de un contenedor
- El **app.py de downloader** actualmente lo que hace es que está esperando que llegue algún mensaje a la cola de rabbitmq
- El **app.py de web-spider** actualmente lo que hace es crear mensajes en formato json
- El **builder.sh** este al ejecutarse sube las imágenes a docker hub
- La carpeta **charts** ya viene lista, hace la ejecución de aplicaciones, es decir, es una herramienta para hacer deployments
 - El **values.yaml** ya viene vacío y así se queda, no se ocupa nada de ahí
 - El **Chart.yaml** especifica metadatos y puede tener dependencias que es el software que queremos instalar, el que viene ahí es para poder usar elasticsearch

- El **install.sh** y **uninstall.sh** son los que instalan y desinstalan los helm charts
- Una vez instaladas los helm charts, en Lens podemos ver los passwords en Config/Secrets
- En Lens, si vamos a Network/Services obtenemos las bases de datos. Cuando elegimos una y le damos a forward nos lleva a la base de datos. El user de elasticsearch es elastic y el password la que tenemos en secrets. El user de rabbit es user
- Desde Lens/Pods podemos acceder a la terminal y logs para ver cómo está funcionando
- El **sample.scala** debemos adaptarlo a lo que nos funcione y sea útil